Movie Night, Part 3: Finishing the Board with Hooks

A step-by-step tutorial on useState, useEffect, useRef, useContext, useMemo and useTransition

---

At the end of Part 2 your Movie Night board worked: ratings, removing and the WATCHED badges all did their job, all of it held in one `useState` call in `App`. In this session you finish it. You add a form that adds movies, load the list from a file the way a real app loads it from a server, add a countdown to showtime, a dark theme that any component can reach, and a search box with sorting that stays fast.

Each new feature needs one of React's built-in **hooks**. By the end you will have used the six covered in the Module 6 notes, each for the job it was designed for.

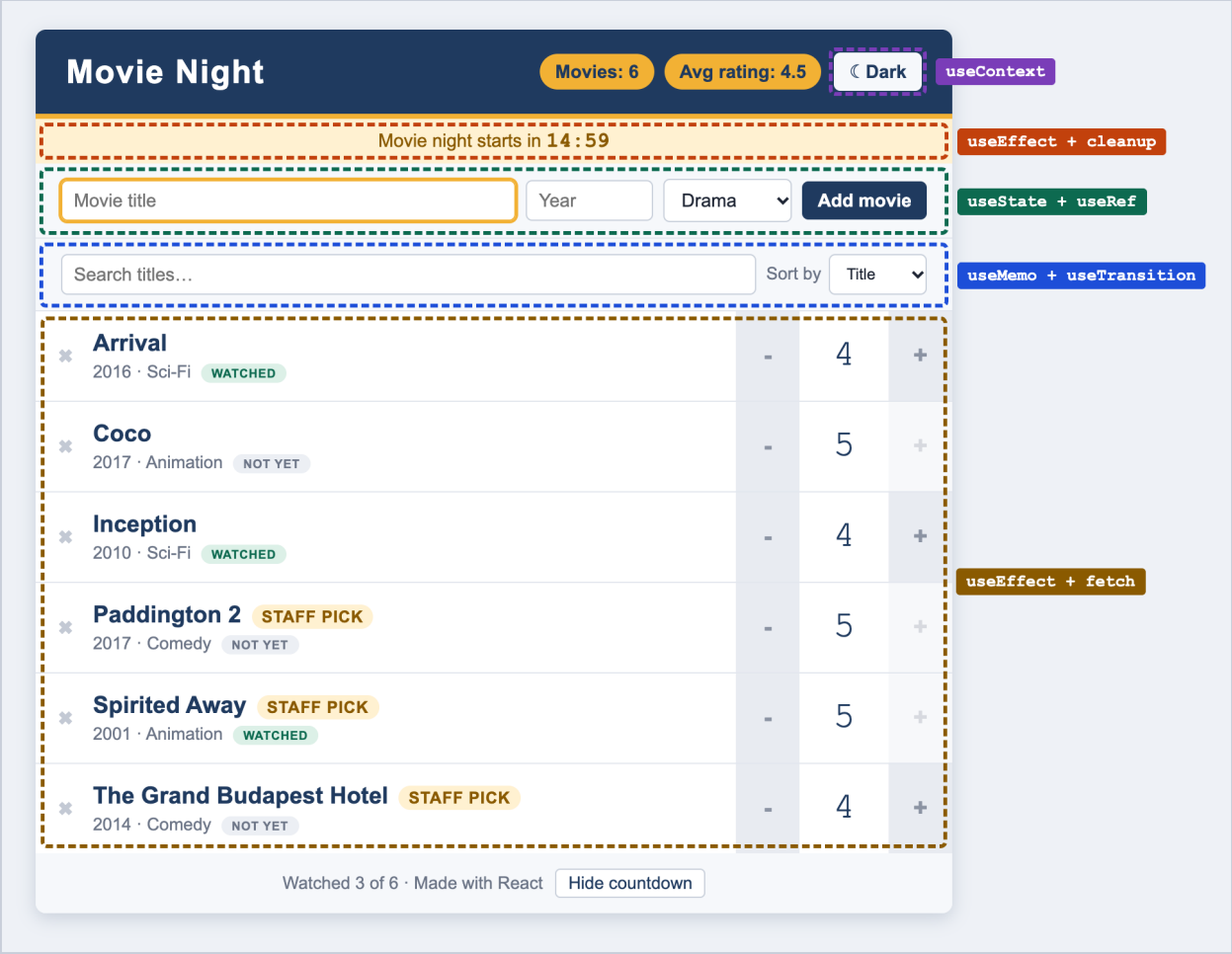


Figure 1 — the finished board, and the hook behind each part of it

> **HOW TO USE THIS HANDOUT**
> Same rules as Parts 1 and 2: your instructor works through each step at the front of the room, and you type the code yourself. Every step ends with a **Checkpoint** folder in `code/checkpoints/`. If you fall behind, copy that folder's `app.js` (and `movies.json`) over yours and carry on.

CONTENTS

Step	Topic	Hook	Minutes
00	Pick up where Part 2 ended, and the rules of hooks	—	8
01	A controlled form that adds movies	`useState`	14
02	Put the cursor where it is needed	`useRef`	8
03	Keep the browser tab in sync	`useEffect`	14
04	Load the movies from a file	`useEffect` + `fetch`	15
05	A countdown that cleans up after itself	`useEffect` cleanup	10
06	Remember without re-rendering	`useRef`	8
07	A theme without prop drilling	`useContext`	15
08	Search and sort, without wasted work	`useMemo`	12
09	Keep typing smooth	`useTransition`	8
10	Wrap-up and self-check		8
	Total		**120**

After the tutorial you will do a 30-minute **Exercise Lab** (a separate handout), which you submit for one mark.

Step 00 — Pick up where Part 2 ended, and the rules of hooks

Goal: a working copy of the finished Part 2 app with the Part 3 files, and the two rules every hook follows.

1. Use your own `movie-night` folder from Part 2. If you don't have it, copy `code/starter/`, which holds the finished Part 2 app.
2. Replace your `app.css` with the one in `code/starter/`. It adds styles for a form, a toolbar, a countdown banner and a dark theme. As before, you do not edit it.
3. Copy `movies.json` from `code/starter/` into your folder. It holds six movies, the four you know plus two new ones. You start using it in Step 04.
4. Open the page with **Live Server**, and open the **Console** and the ⚛ **Components** tab.

Near the top, `app.js` already has the line `const { useState } = React;`. You will add each new hook to that line as you meet it. (In a Vite project you would write `import { useState } from 'react';` instead.)

What is a hook?

A **hook** is a function whose name starts with `use` that lets a *function* component use a React feature: memory between renders, work after rendering, shared values and so on. Here are the six you will use today:

Hook	One-line job	Where it goes in Movie Night
`useState`	Remember a value; changing it re-renders the component	The form's inputs, theme, search text
`useEffect`	Do something *after* React has updated the page	Tab title, loading `movies.json`, the countdown timer
`useRef`	Remember a value **without** re-rendering, or point at a DOM element	Focusing the title box, counting renders
`useContext`	Read a value shared by a parent without passing props by hand	The light/dark theme
`useMemo`	Cache a calculated value until its inputs change	The searched and sorted list
`useTransition`	Mark an update as "can wait", to keep typing responsive	Filtering the list while you type

The two rules of hooks

1. **Only call hooks at the top level of a component** (or of your own custom hook): never inside an `if`, a loop, or a nested function. React tells the hooks of a component apart by the *order* in which they are called, so that order must be the same on every render.

2. **Only call hooks from React function components** (or custom hooks), never from ordinary functions or class components.

```
// WRONG: on some renders this hook is skipped, and every hook after it gets mixed
up
if (movies.length > 0) {
  const [query, setQuery] = useState('');
}

// RIGHT: always call it; put the condition inside your logic instead
const [query, setQuery] = useState('');
```

CHECKPOINT 00
`code/starter/` (plus `movies.json`).

Step 01 — A controlled form that adds movies (useState)

Goal: a form with a title box, a year box and a genre list that adds a movie to the board.

In Part 2 every piece of state lived in `App`. A form is different. What the user is typing belongs to the **form** until they submit, so the form keeps it in its own `useState`. This is **local state**.

1. Above `MovieDetails`, add the `AddMovieForm` component:

```
const AddMovieForm = (props) => {
  const [title, setTitle] = useState('');
  const [year, setYear] = useState('');
  const [genre, setGenre] = useState('Drama');

  const handleSubmit = (e) => {
    e.preventDefault();
    if (title.trim() === '') {
      return;
    }
    props.onAdd({
      title: title.trim(),
      year: Number(year) || new Date().getFullYear(),
      genre: genre
    });
    setTitle('');
    setYear('');
  };
```

```
  return (
    <form className="add-movie" onSubmit={handleSubmit}>
      <input
        type="text"
        placeholder="Movie title"
        value={title}
        onChange={(e) => setTitle(e.target.value)}
      />
      <input
        type="number"
        placeholder="Year"
        value={year}
        onChange={(e) => setYear(e.target.value)}
      />
      <select value={genre} onChange={(e) => setGenre(e.target.value)}>
        <option>Drama</option>
        <option>Comedy</option>
        <option>Animation</option>
        <option>Sci-Fi</option>
        <option>Horror</option>
      </select>
      <button type="submit">Add movie</button>
    </form>
  );
};
```

Each input is **controlled**: its `value` comes from state, and every keystroke calls `onChange`, which updates that state. React re-renders, and the input shows the new value. The state is the single source of truth for what the box contains, which is why `setTitle('')` can clear it after a submit.

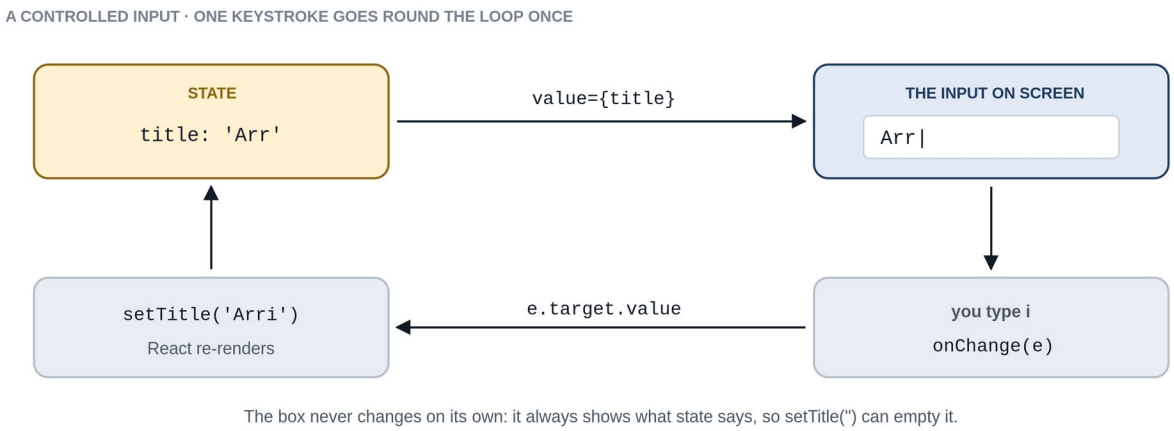


Figure 2 — a controlled input: state feeds value, onChange feeds state

`e.preventDefault()` stops the browser's normal form behaviour, which would reload the page and wipe out all of your state.

2. In `App`, add the handler, next to the other handlers. It is the same pattern as Part 2: a new array, built with the spread operator, and a complete new movie object. `Date.now()` gives a number that is different every millisecond, which is good enough as an id here:

```
const handleAddMovie = (newMovie) => {
  setMovies( prevMovies => [
    ...prevMovies,
    { ...newMovie, rating: 3, watched: false, staffPick: false, id: Date.now() }
  ]);
};
```

3. Render the form in `App`, just under `<Header … />`:

```
    <AddMovieForm onAdd={handleAddMovie} />
```

4. Save. Type `Arrival`, `2016`, choose **Sci-Fi**, and press **Add movie** (or Enter). A fifth row appears, rated 3, and **Movies:** and **Avg rating** update by themselves.

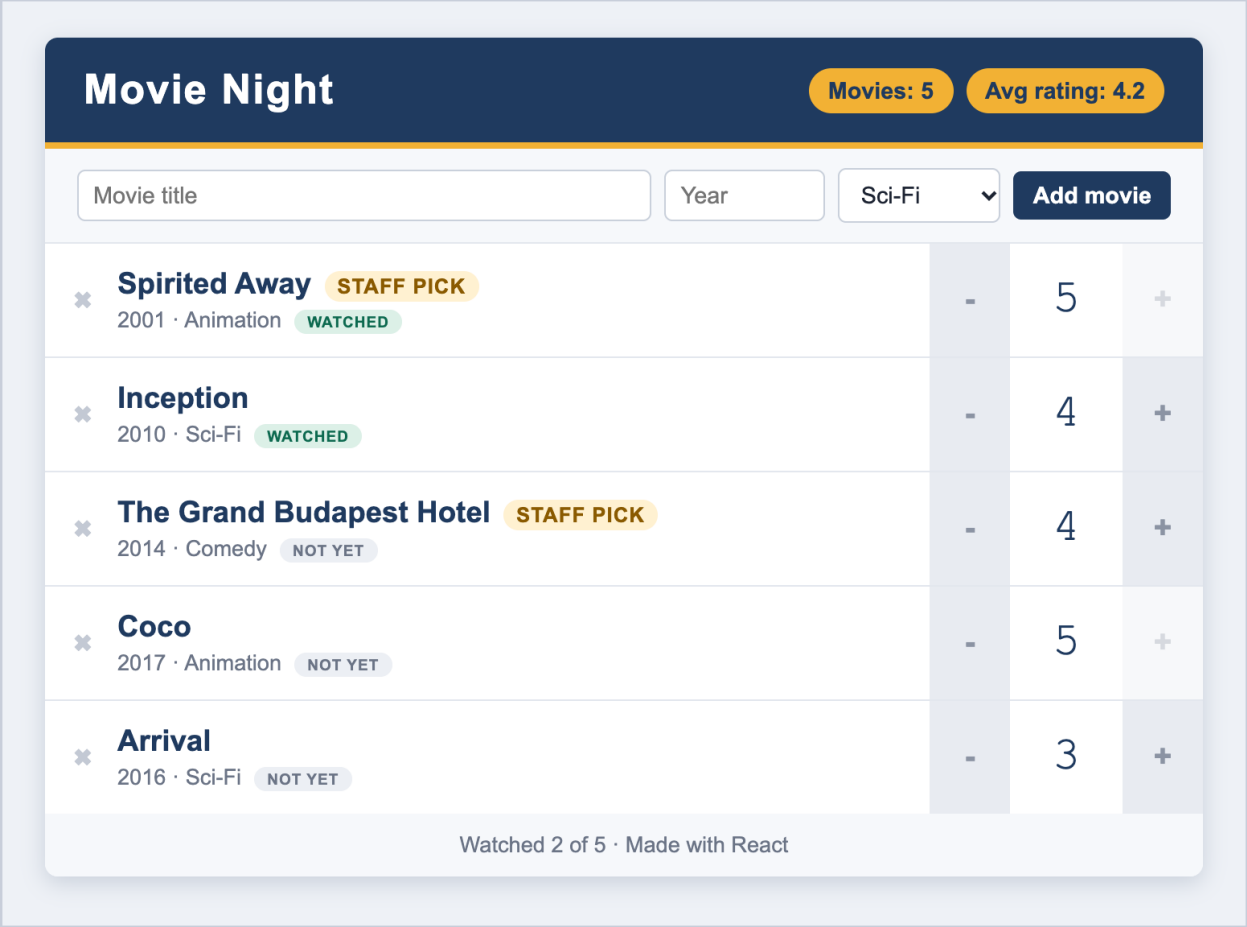


Figure 3 — Arrival added through the form

Notice where things ended up. The **form** owns what is being typed, which is local state. `App` owns the **list**, which is shared state. The form hands a finished movie up through the `onAdd` prop, exactly as ✖ hands an id up in Part 2.

```
CHECKPOINT 01
code/checkpoints/step-01-add-movie-form/
```

Try it (2 min). Click **Add movie** with an empty title. Nothing is added, which is good, but the cursor is still on the button. You have to click back into the title box before you can type again. The next step fixes that.

Step 02 — Put the cursor where it is needed (useRef)

Goal: after adding a movie, or after trying to add a movie without a title, the cursor goes back into the title box.

React builds the page for you, so you rarely touch DOM elements directly. Moving the keyboard focus is one of the few jobs that needs the real element, because there is no `focused` prop to set. **`useRef`** gives you a way to hold on to an element:

1. Add `useRef` to the `const { … } = React;` line: `const { useState, useRef } = React;`
2. In `AddMovieForm`, create a ref under the three `useState` lines:

```
  const titleInput = useRef(null);
```

3. Attach it to the title `<input>` with the special `ref` attribute:

```
      <input
        ref={titleInput}
        type="text"
        …
```

4. Use it in `handleSubmit`, in both places where the user needs to type next:

```
  const handleSubmit = (e) => {
    e.preventDefault();
    if (title.trim() === '') {
      titleInput.current.focus();
      return;
    }
    props.onAdd({ … });
    setTitle('');
    setYear('');
    titleInput.current.focus();
  };
```

`useRef(null)` returns a plain object, `{ current: null }`. When React creates the `<input>`, it puts the DOM element into `titleInput.current`. After that you can call any DOM method on it: `.focus()`, `.select()`, `.scrollIntoView()`.

5. Save. Press **Add movie** with an empty title: the cursor jumps into the title box. Add a movie: the boxes clear and the cursor is ready for the next title.

USE REFS SPARINGLY
A ref is an escape hatch for things React has no prop for: focus, scrolling, measuring an element. Never use one to *change* what is displayed, for example `titleInput.current.value = ''`. That is state's job, and doing it behind React's back makes the state and the screen disagree.

CHECKPOINT 02
`code/checkpoints/step-02-use-ref-focus/`

Step 03 — Keep the browser tab in sync (useEffect)

Goal: the browser tab shows how many movies are watched, for example **Movie Night (2/4 watched)**, and updates whenever that changes.

`document.title` is not part of your JSX. It belongs to the browser, outside React's world. Changing something outside React is called a **side effect**, and side effects do not belong in the body of a component. That body should only *calculate* the JSX. **`useEffect`** runs your code *after* React has finished updating the page.

1. Add `useEffect` to the `const { … } = React;` line: `const { useState, useEffect, useRef } = React;`
2. In `App`, **below** the `watchedCount` / `averageRating` lines (the effect uses them), add:

```
// Keep the browser tab's title in step with the watchlist
useEffect(() => {
  document.title = `Movie Night (${watchedCount}/${movies.length} watched)`;
}, [watchedCount, movies.length]);
```

3. Save and look at the tab: **Movie Night (2/4 watched)**. Click a NOT YET badge and the tab changes to **3/4**.

`useEffect` takes two arguments: the **effect** (a function), and a **dependency array** that tells React when to run it again:

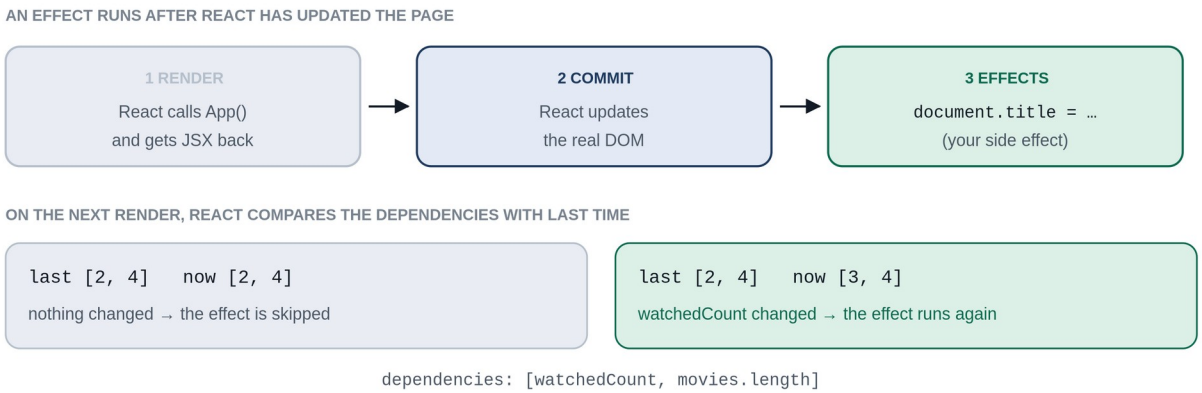


Figure 4 — when an effect runs: after rendering, and again only if a dependency changed

Second argument	The effect runs…	Typical use
(none)	after **every** render	rarely what you want
`[]`	once, after the **first** render	load data, start a timer, add a listener
`[a, b]`	after the first render, then whenever `a` or `b` has changed	keep something in sync with `a` and `b`

4. Prove it. Temporarily add these three effects just above the title effect, then reload:

```
useEffect(() => {
  console.log('A: after every render');
});

useEffect(() => {
  console.log('B: only after the first render');
}, []);

useEffect(() => {
  console.log('C: when watchedCount changes');
}, [watchedCount]);
```

Click **+** on a rating, then click a NOT YET badge:

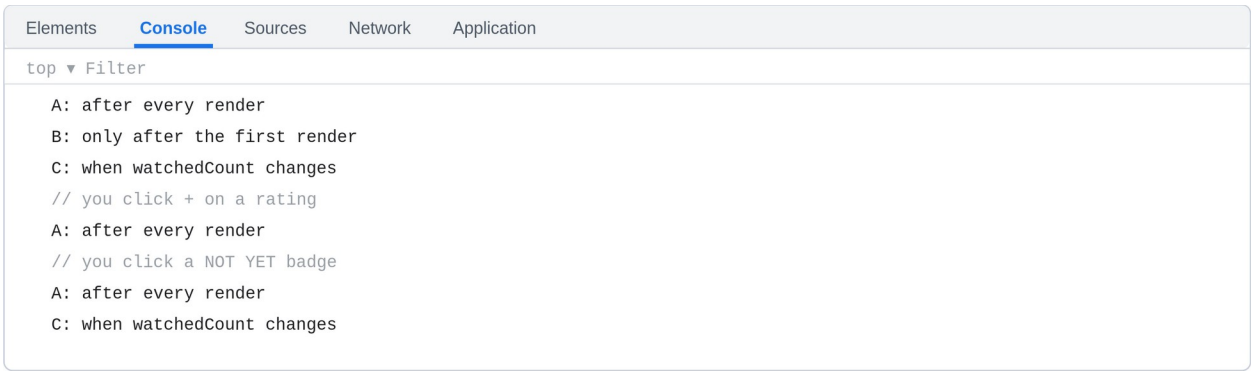


Figure 5 — A runs every time, B only once, C only when the watched count changes

The rating change re-rendered `App` but did not change `watchedCount`, so only **A** ran. The badge changed `watchedCount`, so **C** ran as well. Delete the three experiments afterwards.

5. A second, small effect: make the form put the cursor in the title box as soon as the page opens. In `AddMovieForm`, below the ref:

```
// Put the cursor in the title box when the form first appears
useEffect(() => {
  titleInput.current.focus();
}, []);
```

Effects run after React has put the elements on the page, so `titleInput.current` is ready.

```
CHECKPOINT 03
code/checkpoints/step-03-use-effect-title/
```

Step 04 — Load the movies from a file (useEffect ✦ fetch)

Goal: the movies come from `movies.json`, the way a real app loads them from a server. While they load the user sees **Loading movies…**, and if loading fails they see an error.

So far the movies have been typed into `app.js`. Real apps fetch them. Loading data is a side effect that should happen **once, when the component first appears**: that is `useEffect` with `[]`.

1. Delete the whole `const movies = [ … ];` array from the top of `app.js`. The data now lives in `movies.json`.
2. Below the `const { … } = React;` line, add a small helper that pretends the network is slow, so that you can see the loading message:

```
// Pretends the network is slow, so the loading message stays visible for a moment
const wait = (ms) => new Promise(resolve => setTimeout(resolve, ms));
```

3. `App` no longer receives the movies as a prop. Change `const App = (props) => {` to `const App = () => {`, and replace its first line with **three** pieces of state: the data, and whether you are still waiting for it or it failed:

```
  const [movies, setMovies] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
```

4. Add the loading effect right under them:

```
  // Load the movies once, when App first appears
  useEffect(() => {
    const loadMovies = async () => {
      try {
        await wait(800);
        const response = await fetch('./movies.json');
        if (!response.ok) {
          throw new Error(`HTTP ${response.status}`);
        }
        const data = await response.json();
        setMovies(data);
      } catch (err) {
        setError(`Could not load the movies (${err.message})`);
      } finally {
        setLoading(false);
      }
    };

    loadMovies();
  }, []);
```

This is the Module 3 `fetch` pattern inside an effect. Note the shape: the effect function itself is **not** `async`. An effect may only return a cleanup function (Step 05), and an `async` function always returns a promise. So you define an `async` function *inside* the effect and call it. If you write `useEffect(async () => …)`, React prints a warning explaining exactly this.

5. Show the two waiting states just above the `map()` in `App`'s JSX, and replace Part 2's empty-list message with one that only appears once loading has finished:

```
      {/* Movie list */}
      {loading && <p className="status">Loading movies…</p>}
      {error && <p className="status status-error">{error}</p>}
      {!loading && !error && movies.length === 0 &&
        <p className="empty">No movies to show.</p>}
```

6. At the bottom of the file, the movies are no longer passed in: `root.render(<App />);`
7. Save. For a moment you see **Loading movies…**, then six movies appear.

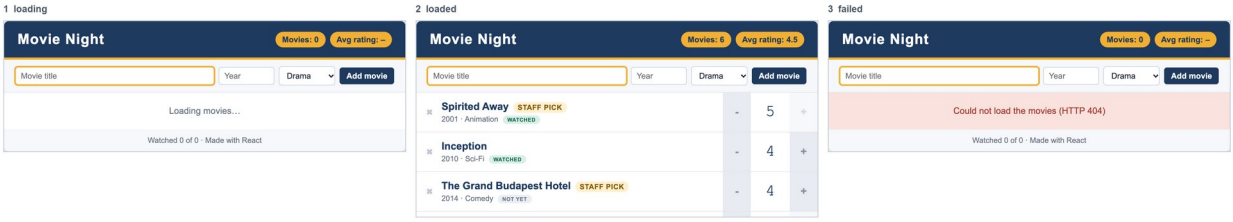


Figure 6 — the three states of a page that loads data

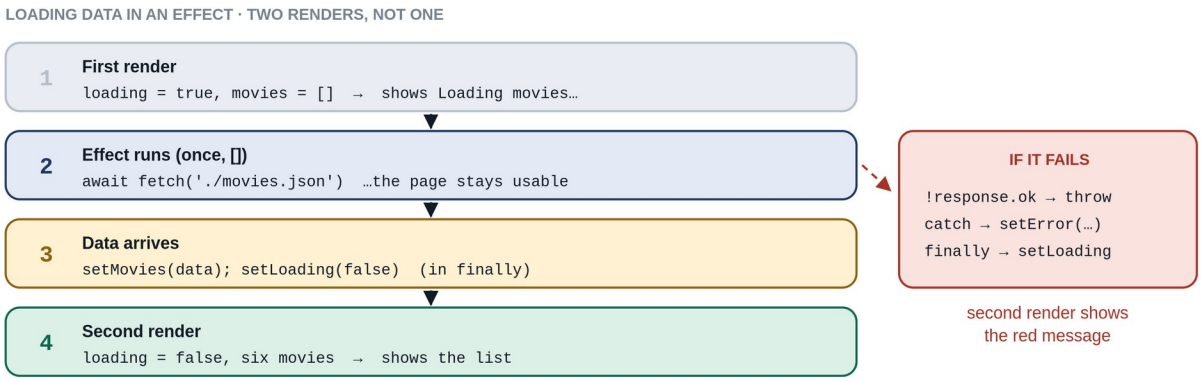


Figure 7 — what happens, in order, when App first appears

8. Test the error path. Change `'./movies.json'` to `'./movie.json'` and save: the red message reads **Could not load the movies (HTTP 404)**. `fetch` does not fail on a 404 by itself (you met this in Module 3), which is why the code checks `response.ok`. Change it back.

> **WHY [] MATTERS HERE**
> Without the dependency array, the effect would run after *every* render. `setMovies` causes a render, which runs the effect, which fetches again and calls `setMovies`… The page would download `movies.json` over and over, forever.

> **CHECKPOINT 04**
> `code/checkpoints/step-04-fetch-movies/` (needs `movies.json` beside it)

Step 05 — A countdown that cleans up after itself (useEffect cleanup)

Goal: a banner reading **Movie night starts in 14:59** that counts down every second, and that can be hidden and shown from the footer without leaving a timer running.

1. Add a `Countdown` component above `AddMovieForm`. It owns its own `secondsLeft` state, and an effect starts a timer when it appears:

```
// mm:ss, for example 14:59
const formatTime = (totalSeconds) => {
  const minutes = Math.floor(totalSeconds / 60);
  const seconds = totalSeconds % 60;
  return `${String(minutes).padStart(2, '0')}:${String(seconds).padStart(2, '0')}`;
};

const Countdown = (props) => {
  const [secondsLeft, setSecondsLeft] = useState(props.startSeconds);

  useEffect(() => {
    const intervalId = setInterval(() => {
      setSecondsLeft(prevSeconds => Math.max(prevSeconds - 1, 0));
    }, 1000);

    // Cleanup: React runs this when the Countdown is removed from the page
    return () => {
      clearInterval(intervalId);
      console.log('Countdown stopped');
    };
  }, []);

  return (
    <p className="countdown">
      { secondsLeft > 0
          ? <>Movie night starts in <strong>{ formatTime(secondsLeft) }</strong></>
          : <strong>Showtime!</strong> }
    </p>
  );
};
```

The function an effect **returns** is its **cleanup**. React calls it when the component disappears from the page (and also before re-running an effect whose dependencies changed). Any effect that starts something that keeps going on its own, such as a timer, an event listener or a subscription, should stop it in the cleanup.

Note `setSecondsLeft(prevSeconds => …)`. The timer's function was created once, on the first render, so a plain `secondsLeft` inside it would be stuck at its first value forever. The function form always receives the latest value (Part 2, Step 07).

2. In `App`, add a piece of state for whether the banner is showing, and render it under the header:

```
const [showCountdown, setShowCountdown] = useState(true);
```

```
      {showCountdown && <Countdown startSeconds={15 * 60} />}
```

3. Give the `Footer` a button that toggles it. In `App`, pass two more props to `<Footer … />`:

```
        showCountdown={showCountdown}
        toggleCountdown={() => setShowCountdown( prev => !prev )}
```

…and in `Footer`, add the button after the text:

```
    <button onClick={props.toggleCountdown}>
      { props.showCountdown ? 'Hide countdown' : 'Show countdown' }
    </button>
```

4. Save. The banner counts down. Click **Hide countdown**: the banner disappears and the Console prints **Countdown stopped**. Click **Show countdown**: a brand-new `Countdown` appears, starting again from 15:00, because its state was thrown away when it was removed.

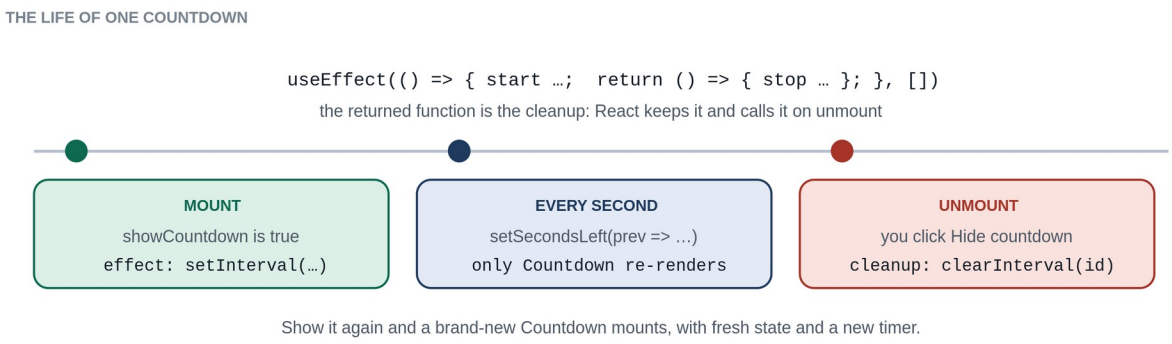


Figure 8 — mount starts the timer; unmount runs the cleanup and stops it

5. See why the cleanup matters. Temporarily delete the `return () => { … };` block, add `console.log('tick');` as the first line inside the `setInterval` function, save, and click **Hide countdown**.

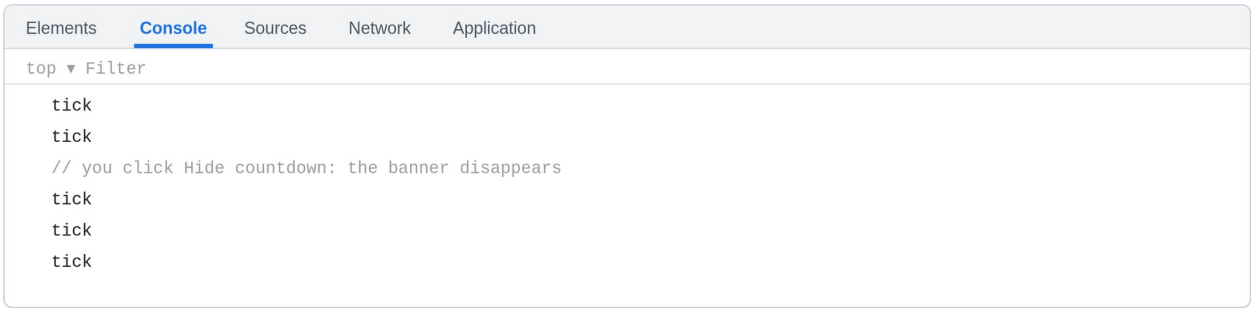


Figure 9 — without a cleanup, the hidden countdown's timer keeps ticking

The banner has gone, but **tick** keeps printing. The timer is still running, for a component that no longer exists. Show and hide it a few times and several timers tick at once. This kind of bug is called a **memory leak**. Put the cleanup back and remove the `tick` line.

> **IN A VITE PROJECT YOU WILL SEE EFFECTS RUN TWICE**
> In development, React's `StrictMode` (which Vite turns on) mounts every component, removes it, and mounts it again, to check that your cleanups work. You would see **Countdown stopped** once at startup. That is expected, it happens in development only, and it is harmless when the cleanup is correct.

> **CHECKPOINT 05**
> `code/checkpoints/step-05-countdown-cleanup/`

Step 06 — Remember without re-rendering (useRef)

Goal: count how many times `App` renders, and see the second job of `useRef`.

A ref is not only for DOM elements. `useRef(0)` is a box, `{ current: 0 }`, that React keeps for the whole life of the component, like state. Unlike state, changing `.current` does **not** cause a re-render.

1. In `App`, under the `useState` lines:

```
const renderCount = useRef(0);
```

2. And an effect without a dependency array, so it runs after every render:

```
// Runs after every render. Changing a ref does not cause another render.
useEffect(() => {
  renderCount.current = renderCount.current + 1;
  console.log(`App has rendered ${renderCount.current} times`);
});
```

3. Reload and watch the Console. You get **1** (the first render, still loading), then **2** (the movies arrived). Click **+** on a rating: **3**. Now watch the countdown tick. The Console stays silent, because the countdown's state lives in `Countdown`, so only `Countdown` re-renders each second. `App` does not.
4. Why not use state for the counter? Think it through (do not try it in class, as it freezes the tab). With `const [renders, setRenders] = useState(0)` and `setRenders(renders + 1)` in an effect that runs after every render, updating the count causes a render, the render runs the effect, and the effect updates the count again. React stops it with **Maximum update depth exceeded**, after tens of thousands of renders. A ref can count renders precisely *because* changing it does not cause one.

	`useState`	`useRef`
Survives re-renders	Yes	Yes
Changing it re-renders the component	**Yes**	**No**
How you change it	The setter:	Assign: `ref.current = …`

	`useState`	`useRef`
	`setValue(…)`	
Use it for	Anything shown on screen	DOM elements, timer ids, counters and values the screen does not depend on

> **DO NOT SHOW A REF'S VALUE IN THE JSX**
> `{renderCount.current}` would display a number that is one render behind, because the effect updates it *after* rendering. If the screen needs a value, that value is state.

> **CHECKPOINT 06**
> `code/checkpoints/step-06-use-ref-render-count/`

Step 07 — A theme without prop drilling (useContext)

Goal: a ☾ Dark / ✳ Light button in the header switches the whole board, including the rating controls, between a light and a dark theme.

Step 07a — First, the way you already know: props

1. In `App`, add the theme state and a toggle:

```
  const [theme, setTheme] = useState('light');
```

```
  const toggleTheme = () => {
    setTheme( prevTheme => prevTheme === 'light' ? 'dark' : 'light' );
  };
```

2. Put the theme on the card, so the stylesheet can colour everything inside it:

```
      <div className={`watchlist theme-${theme}`}>
```

3. The header needs the button, so pass `theme={theme}` and `toggleTheme={toggleTheme}` to `<Header … />`, and add this inside `Header`, after the second `stats` pill:

```
<button className="theme-toggle" onClick={props.toggleTheme}>
  { props.theme === 'light' ? '☾ Dark' : '☀ Light' }
</button>
```

4. `Rating` needs the theme too, to colour its buttons. But `Rating` sits inside `Movie`, so the value has to travel **through** `Movie`: pass `theme={theme}` to every `<Movie … />` in `App`'s `map()`, pass `theme={props.theme}` from `Movie` to `<Rating … />`, and in `Rating` write:

```
<div className={`rating rating-${props.theme}`}>
```

5. Save and click **☾ Dark**. It works:

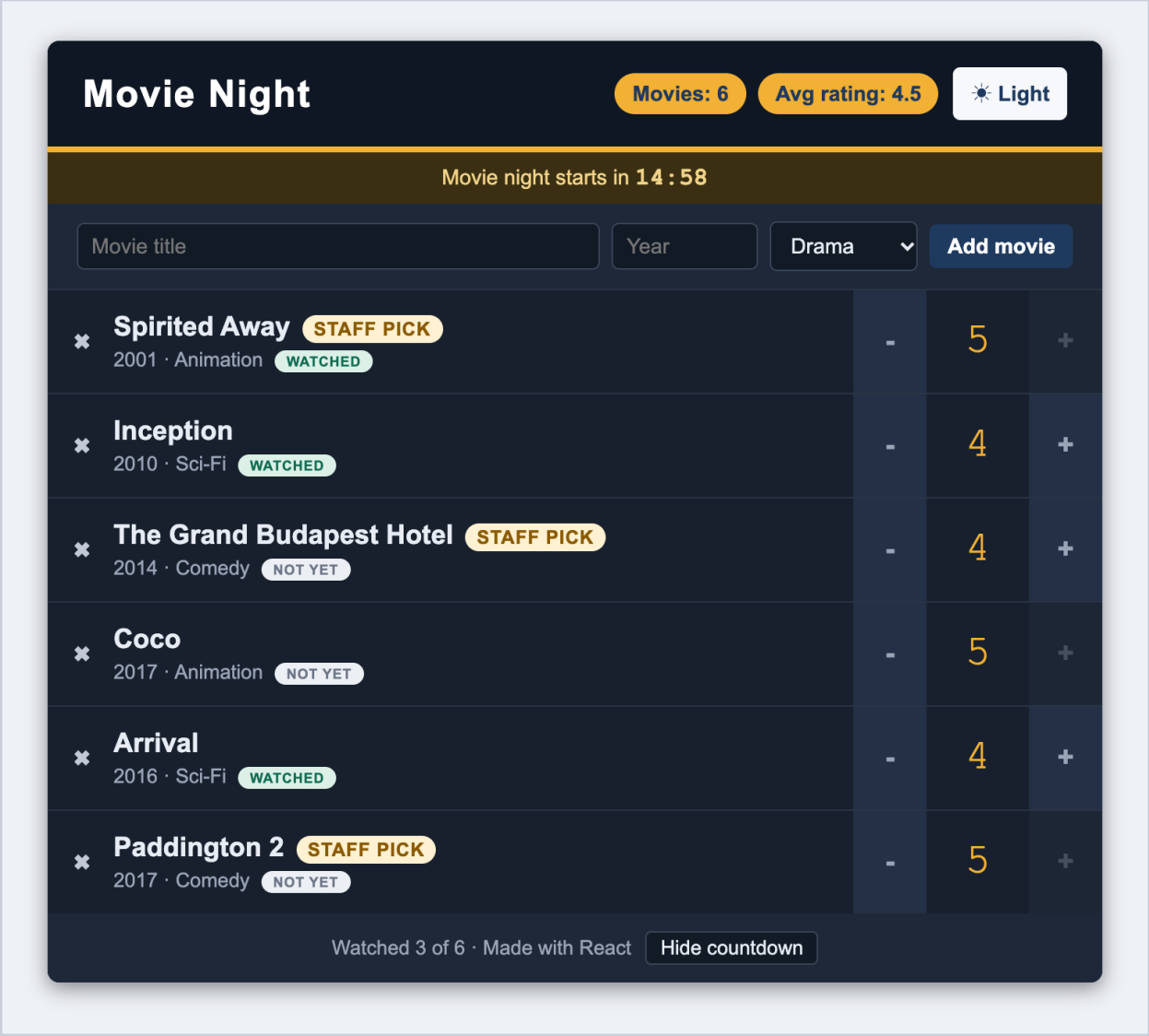


Figure 10 — the dark theme

But look at what you did to `Movie`. It does not use `theme` at all. It only receives it in order to pass it on. That is **prop drilling**. With one value and one level it is a small nuisance. With a signed-in user, a language and a currency, passed down five levels, every component in the middle ends up full of props it does not care about.

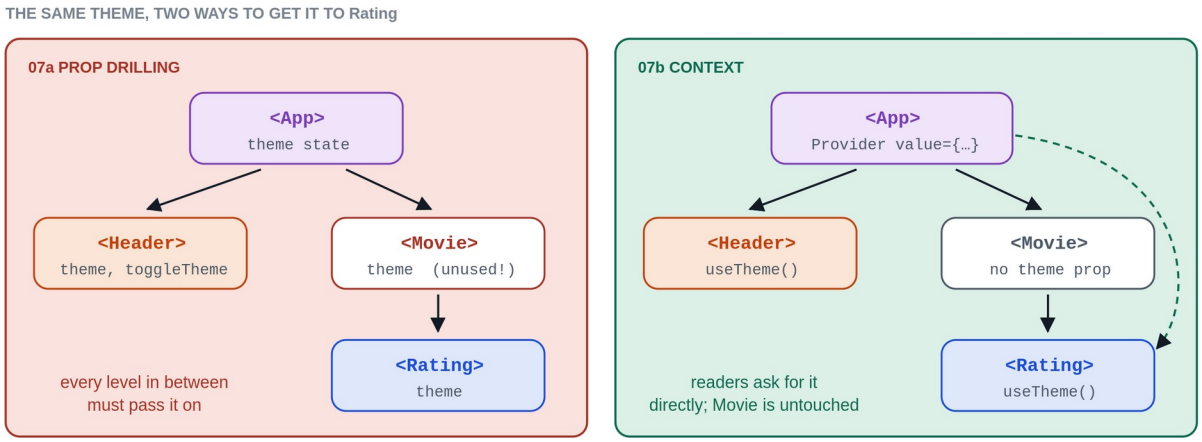


Figure 11 — prop drilling: Movie only passes theme along; context lets Rating read it directly

```
CHECKPOINT 07A
code/checkpoints/step-07a-prop-drilling/
```

Step 07b — The same, with context

Context lets a parent make a value available to every component below it, without passing props by hand. It takes three parts: **create** a context, **provide** a value high in the tree, and **consume** it wherever it is needed.

1. Add `useContext` and `createContext` to the `const { … } = React;` line.
2. **Create** the context, near the top of the file (below the helpers), and a small **custom hook** that reads it:

```
const ThemeContext = createContext(null);

// A custom hook: a shorter, safer way for any component to read the theme
const useTheme = () => {
  const context = useContext(ThemeContext);
  if (context === null) {
    throw new Error('useTheme must be used inside a ThemeContext.Provider');
  }
  return context;
};
```

A **custom hook** is just a function whose name starts with `use` and that calls other hooks. This one saves every component from importing `ThemeContext` itself, and it gives a clear error if someone uses it outside the provider, where the context value would be `null`.

3. **Provide** the value in `App`. Wrap everything that `App` returns in a provider, and pass it one object holding both the value and the function that changes it:

```
  const themeValue = { theme: theme, toggleTheme: toggleTheme };

  return (
    <ThemeContext.Provider value={themeValue}>
      <div className={`watchlist theme-${theme}`}>
        …everything that was there before…
      </div>
    </ThemeContext.Provider>
  );
```

4. **Consume** it. Remove every `theme` and `toggleTheme` prop you added in 07a: from `<Header>`, from `<Movie>` in the `map()`, and from `<Rating>` in `Movie`. Then let `Header` and `Rating` ask for the theme themselves:

```
const Header = (props) => {
  const { theme, toggleTheme } = useTheme();

  return (
    <header>
      …
      <button className="theme-toggle" onClick={toggleTheme}>
        { theme === 'light' ? '☾ Dark' : '☀ Light' }
      </button>
    </header>
  );
};
```

```
const Rating = (props) => {
  const { theme } = useTheme();

  return (
    <div className={`rating rating-${theme}`}>
      …
```

5. Save. The theme works exactly as before, but `Movie` is back to its Part 2 code. In ⚛ **Components**, click the `Context.Provider` entry at the top of the tree to see the value it is passing down.

When the provider's `value` changes, React re-renders **every component that reads that context**, however deep it sits. Components in between are left alone unless they have their own reason to re-render.

> **WHEN TO USE CONTEXT, AND WHEN NOT TO**
> Use it for values that many components at different depths need: the theme, the signed-in user, the language. For a value that a parent passes to its direct child, keep using props: they make the data flow easy to follow. Keep each context focused on one concern. A `ThemeContext` and a `UserContext` are better than one `AppContext` that holds everything.

> **CHECKPOINT 07B**
> `code/checkpoints/step-07b-use-context-theme/`

Step 08 — Search and sort, without wasted work (useMemo)

Goal: a search box and a **Sort by** list above the movies. The filtered and sorted list is recalculated only when it needs to be.

1. Add a `Toolbar` component above `MovieDetails`. It keeps no state: its values come from `App`, and it reports changes up, as in Part 2:

```
const Toolbar = (props) => {
  return (
    <div className="toolbar">
      <input
        type="search"
        placeholder="Search titles…"
        value={props.query}
        onChange={(e) => props.onQueryChange(e.target.value)}
      />
      <label>
        Sort by
        <select value={props.sortBy} onChange={(e) =>
props.onSortChange(e.target.value)}>
          <option value="title">Title</option>
          <option value="rating">Rating</option>
          <option value="year">Newest</option>
        </select>
      </label>
    </div>
  );
};
```

2. In `App`, add the two pieces of state, and render the toolbar under the form:

```
  const [query, setQuery] = useState('');
  const [sortBy, setSortBy] = useState('title');
```

```
        <Toolbar
          query={query}
          onQueryChange={setQuery}
          sortBy={sortBy}
          onSortChange={setSortBy}
        />
```

3. Imagine the filtering is expensive: thousands of movies, or a fuzzy search. This helper burns time on purpose so that you can *feel* the cost. Add it below `wait`:

```
// Burns CPU time on purpose: a stand-in for an expensive calculation
const slowDown = (ms) => {
  const start = performance.now();
  while (performance.now() - start < ms) {
    // do nothing
  }
};
```

4. Calculate the visible list in `App`, **without** `useMemo` first, and use it everywhere the list is rendered. That means the `map()` and the "No movies to show" check:

```
  const visibleMovies = (() => {
    console.log('Filtering and sorting the movies…');
    slowDown(150);
    const text = query.toLowerCase();
    return movies
      .filter( movie => movie.title.toLowerCase().includes(text) )
      .sort( (a, b) =>
        sortBy === 'rating' ? b.rating - a.rating
        : sortBy === 'year' ? b.year - a.year
        : a.title.localeCompare(b.title)
      );
  })();
```

`.filter()` returns a **new** array, so `.sort()` (which rearranges an array in place) sorts the copy and never touches `movies` in state. The **Header**, the **Footer** and the tab title still use `movies`: the counts describe the whole watchlist, not just the search results.

5. Save. Type `co` in the search box, then switch **Sort by** to **Rating**. Both work. Now click **☾ Dark** a few times and watch the Console: **Filtering and sorting the movies…** prints on *every* click, and the button feels sluggish. Switching the theme re-renders `App`, and the whole calculation runs again, although the theme has nothing to do with which movies are shown.
6. Fix it with **`useMemo`**. Add it to the `const { … } = React;` line, and wrap the calculation. The function stays the same; it moves inside `useMemo`, with a dependency array:

```
// Only recalculated when movies, query or sortBy change
const visibleMovies = useMemo(() => {
  console.log('Filtering and sorting the movies…');
  slowDown(150);
  const text = query.toLowerCase();
  return movies
    .filter( movie => movie.title.toLowerCase().includes(text) )
    .sort( (a, b) =>
      sortBy === 'rating' ? b.rating - a.rating
      : sortBy === 'year' ? b.year - a.year
      : a.title.localeCompare(b.title)
    );
}, [movies, query, sortBy]);
```

7. Click **☾ Dark** again: no log, and the button is instant. Type in the search box or change the sort: the log appears, because those *are* dependencies.

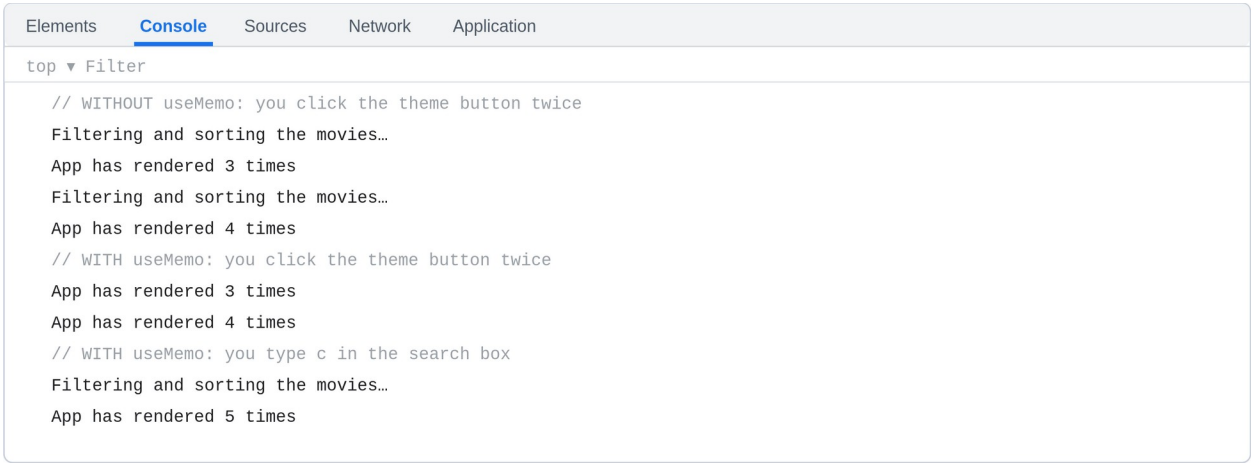


Figure 12 — useMemo returns the cached list until a dependency changes

`useMemo(calculate, [deps])` runs `calculate` on the first render and stores the result. On later renders it compares each dependency with its previous value. If none has changed, it returns the stored result **without** running `calculate`.

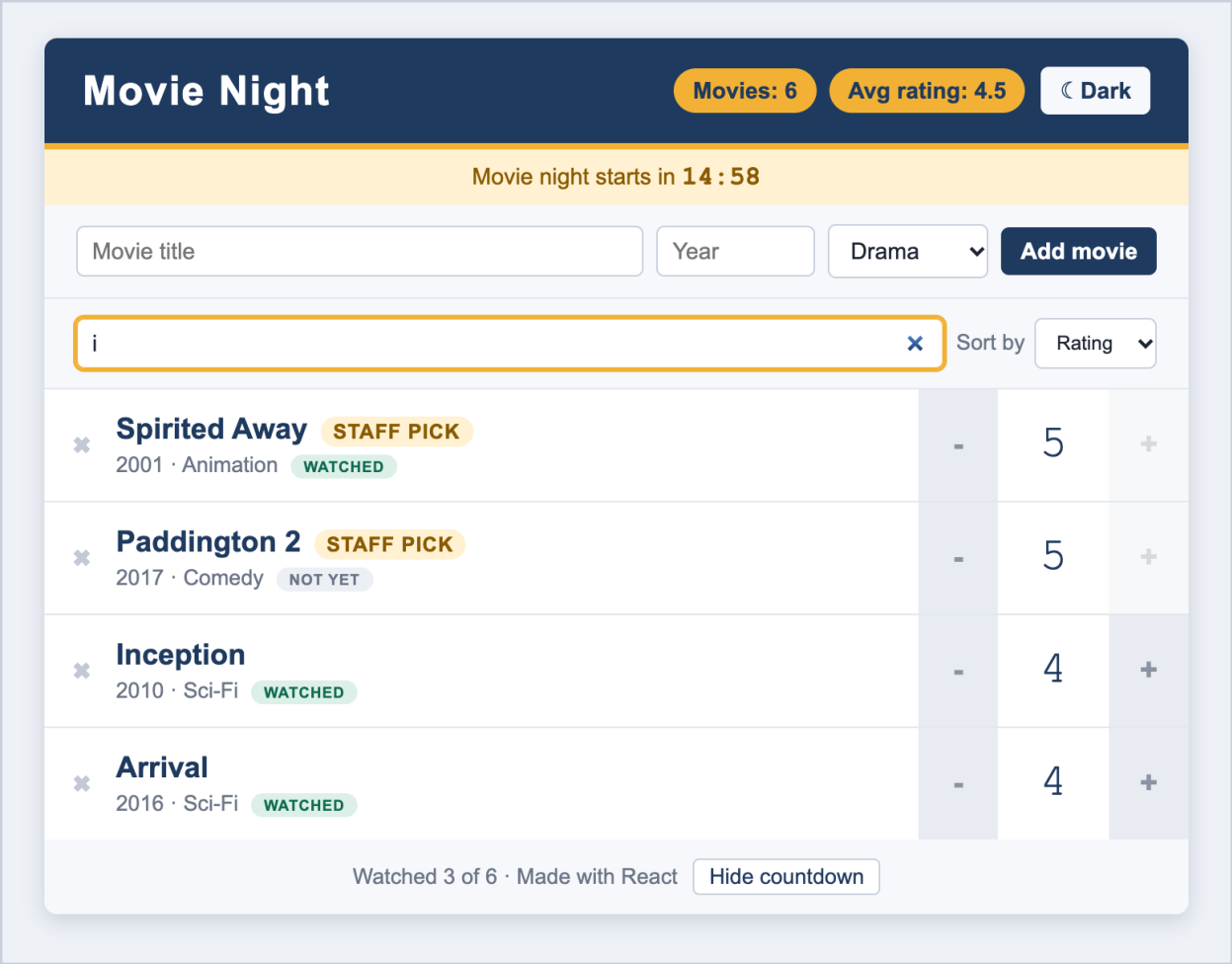


Figure 13 — searching for i, sorted by rating

8. One more `useMemo`. Every render of `App` creates a brand-new `themeValue` object, even when the theme has not changed, and a new object tells every `useTheme()` reader that the context changed. Keep the same object until `theme` changes. Replace `toggleTheme` and `themeValue` with:

```
  // The same object until theme changes, so context readers are not disturbed
  const themeValue = useMemo(() => ({
    theme: theme,
    toggleTheme: () => setTheme( prevTheme => prevTheme === 'light' ? 'dark' :
'light' )
  }), [theme]);
```

> **DO NOT MEMO EVERYTHING**
> `useMemo` has its own cost: it stores values and compares dependencies on every render. Use it for calculations that are genuinely slow, or for objects you pass to many readers (like a context value). `watchedCount` is a quick count of six items, so it stays a plain `const`. `useMemo` caches a **value**. It does not stop the component from re-rendering.

> **CHECKPOINT 08**
> `code/checkpoints/step-08-use-memo-search-sort/`

Step 09 — Keep typing smooth (useTransition)

Goal: while the list is being filtered, the letters you type still appear immediately.

Type quickly in the search box. Each letter appears only after the slow filtering has finished, so typing feels sticky. One keystroke causes two jobs: an **urgent** one (show the letter in the box) and one that **can wait** a moment (filter the list). Right now React does both in the same render, so the urgent one waits for the slow one.

`useTransition` lets you mark an update as a **transition**, meaning not urgent. React first renders the urgent update, so the letter appears, and then works on the transition.

1. Add `useTransition` to the `const { … } = React;` line.
2. You need two values now: what is **in the box** (urgent) and what the list is **filtered by** (can wait). In `App`, keep `query` and add:

```
const [filterText, setFilterText] = useState('');
const [isPending, startTransition] = useTransition();
```

3. Add a handler that updates both, the second one inside `startTransition`:

```
const handleQueryChange = (text) => {
  setQuery(text);                  // urgent: the letters in the box
  startTransition(() => {
    setFilterText(text);           // not urgent: the filtered list
  });
};
```

4. In the `useMemo`, filter by `filterText` instead of `query`: change `const text = query.toLowerCase();` to `const text = filterText.toLowerCase();`, and change `query` to `filterText` in its dependency array.
5. Pass the new handler and `isPending` to the toolbar:

```
      <Toolbar
        query={query}
        onQueryChange={handleQueryChange}
        sortBy={sortBy}
        onSortChange={setSortBy}
        isPending={isPending}
      />
```

6. In `Toolbar`, show a hint while the transition is in progress, after the `</label>`:

```
      { props.isPending && <span className="pending">Updating list…</span> }
```

7. Save and type `c`. React now renders **twice**. First, the urgent render: the box shows `c`, `isPending` is `true`, and the old list is still on screen. Then the transition render: the list is filtered and the hint disappears.

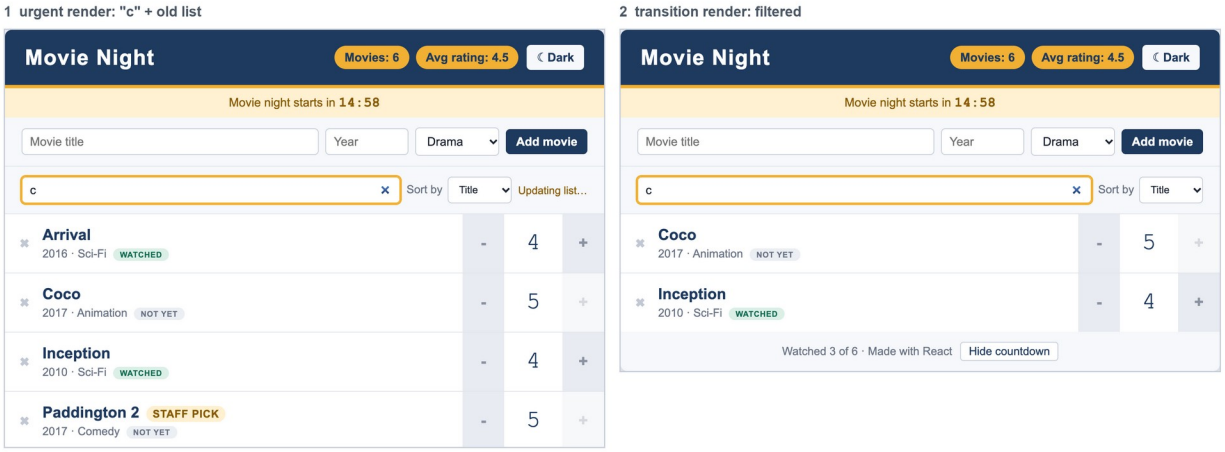


Figure 14 — one keystroke, two renders: the letter first, the filtered list after

With only six movies the hint flashes for a moment. With the `slowDown` raised to `600`, it stays long enough to read. Try it, then set it back to `150`.

> **TRANSITIONS CHANGE PRIORITY, NOT SPEED**
> The filtering takes exactly as long as before. What changes is the *order*: urgent feedback first. Never put the input's own `value` update in a transition, or the box itself would lag. Small apps rarely need `useTransition`; it pays off with large lists, heavy tables and dashboards.

> **CHECKPOINT 09 — THE FINISHED APP**
> `code/checkpoints/step-09-use-transition-final/`

Step 10 — Wrap-up and self-check

What you built, and the hook behind each part

Feature	Hook	The idea
Add-movie form	`useState` (local)	Controlled inputs: state is the single source of truth for what a box contains
Cursor back in the title box	`useRef` + `ref`	A ref gives you the real DOM element for jobs React has no prop for
Tab title	`useEffect` with `[deps]`	Effects sync React with the outside world after rendering
Movies loaded from `movies.json`	`useEffect` with `[]` + `fetch`	Load once on mount; show loading and error states
Countdown	`useEffect` + cleanup	Whatever an effect starts, its cleanup must stop
Render counter	`useRef`	A ref remembers without causing a render
Dark theme	`createContext` + `useContext` + custom hook	Share a value with any depth of component, without prop drilling
Search and sort	`useMemo`	Cache an expensive value until its inputs change
Smooth typing	`useTransition`	Urgent updates first; the slow ones can wait

Self-check

1. State the two rules of hooks, and explain why the first one exists.

2. What makes an input "controlled"? Why can `setTitle('')` clear the box?
3. What does `useRef(null)` return, and when is `titleInput.current` filled in?
4. When does an effect with no array run? With `[]`? With `[watchedCount]`?
5. Why is the effect function in Step 04 not `async` itself?
6. What happens to a `setInterval` started in an effect if the component is removed and there is no cleanup?
7. Give one difference between `useState` and `useRef`, and one value from today that belongs in each.
8. What is prop drilling? Which component did it affect in Step 07a?
9. What are the three parts of using context?
10. Clicking ☾ Dark printed **Filtering and sorting...** before `useMemo`, and not after. Why?
11. In Step 09, why are there two pieces of state (`query` and `filterText`) for one search box?

Answer key

1. Only call hooks at the top level of a component or custom hook, and only from function components or custom hooks. React matches each hook to its stored value by call order, so the order must be identical on every render. A hook inside an `if` could be skipped and shift every hook after it.
2. Its `value` comes from state, and `onChange` updates that state on every keystroke. The box always displays the state, so setting the state to `''` empties the box.
3. An object `{ current: null }`. React fills `current` with the DOM element when it puts the `<input>` on the page, so it is ready inside event handlers and effects.
4. No array: after every render. `[]`: once, after the first render. `[watchedCount]`: after the first render, and after any render in which `watchedCount` changed.
5. An effect may only return nothing or a cleanup function. An `async` function always returns a promise, so the async work goes in an inner function that the effect calls.
6. It keeps running forever, for a component that no longer exists (a memory leak). Showing and hiding the component starts more and more timers.
7. Changing state re-renders the component; changing `ref.current` does not. State: the form's inputs, the theme, the search text. Ref: the title input element, the render count.
8. Passing a prop through a component that does not use it, only to reach a deeper one. `Movie` received `theme` only to pass it to `Rating`.

9. Create (`createContext`), provide (`<ThemeContext.Provider value={…}>`), consume (`useContext(ThemeContext)`, here wrapped in the `useTheme` custom hook).
10. Changing the theme re-renders `App`. Without `useMemo` the calculation runs on every render. With it, React sees that `movies`, `query` and `sortBy` are unchanged and returns the cached list.
11. They have different priorities. `query` drives the box, and must update immediately. `filterText` drives the slow list, and is updated in a transition so that typing never waits for it.